README: Project 1. Socket Programming Practice

Student ID: 2021-18641

Name: Hadong Lee

1. Implementation Strategy

1.0 Common Logic

- For reusability, the functions read_all and write_all were extracted, which use the system-call level read and write.

- A function called skip_blank was defined to handle an arbitrary number of whitespace characters by calling isblank.

- Standard string functions such as strncmp, strstr, and strncasecmp are utilized for string parsing.

1.1 Implementation Strategy of sclient

- Since the response format from the server is assumed to always be well-formed, the parsing logic is kept simple; only the response code is parsed.

- A buffer of up to 10 MB is required, so a buffer is allocated on the heap.

- An error label is placed at the end, and goto error is used for concise error handling.

- The overall flow is as follows: Connect to the server → Read the body from stdin and calculate content-length → Construct the header → Send both the header and the body to the server.

- The call to shutdown(socketfd, SHUT_WR); is used to explicitly send an EOF after writing the request to the server, thereby allowing the client to switch to reading mode.

- Errors are handled consistently by printing an error message and terminating the client.

1.2 Implementation Strategy of sserver

- To support concurrency, a multi-process strategy was adopted; 6 child processes are created so that each calls accept to handle up to 6 requests concurrently.

- The main function sets up server_fd to listen and, after forking, only contains logic to wait for all

child processes via waitpid. The actual request processing logic is encapsulated in a separate function called child_loop, which takes the listen_fd as an argument. This design aims easier modifications to the concurrency strategy in the future.

- Whenever interrupt or child termination detected, parent process sends kill to remaining children and terminate.

- The child_loop logic is as follows: Accept a client connection → Read the entire request from the client_fd and locate CRLFCRLF to separate the header and body → Parse the header and use the content-length from the header to send back the header and body to the client.

- If the body is larger than the declared content-length, the excess is discarded; if it is smaller, the client's request is considered abnormal and a 400 Bad Request is returned.

- The parsing logic works by:

    1. Parsing the first line using skip_blank along with successive calls to strncmp for each token → Assuming only the Host and Content-length headers are present, an if-statement checks whether the first header is Host or Content-length, and then parses each line sequentially.

    2. After parsing the two header lines, an empty line is expected to ensure that no additional unnecessary headers exist.

    3. Errors in parsing return an error code, which is checked in the processing loop; if an error is detected, the connection is closed and the loop continues to the next connection.

2. Program Test

Two aspects were tested:

- Whether the I/O functions properly with a sufficiently large body.
- Whether the header parsing works correctly even when headers include multiple spaces or mixed-case characters.

Testing Procedure:

- A 10 MB text file was prepared and sent as a request. The response body received by the client was saved into a separate text file, and a diff was used to verify proper processing.
- A test client was used to send various headers (with extra spaces, different casing, etc.) to ensure that the header parsing logic is robust.

3. Known Bugs and Weaknesses

- Limited Extensibility in Header Parsing:

The current parsing functions assume a fixed order and only handle the Host and Content-length headers. A future improvement would be to adopt a flag-based approach to handle additional headers.

- Scalability and Buffer Size Issues in Process-Based Concurrency:

Each child process allocates a 10 MB buffer regardless of the request size, which may lead to inefficiencies for smaller requests. It might need to be changed using dynamic allocation using realloc in later.

- Signal-Based Child Process Termination:

Although the OS reclaims memory when a process terminates (thus avoiding memory leaks), the signal-based termination means that buffers are not explicitly freed. This approach is acceptable for this assignment but is not ideal.

4. References

2024 Fall System Programming HW5

2025 Spring Network Slide 2

Used man pages and GPT to understand standard C functions